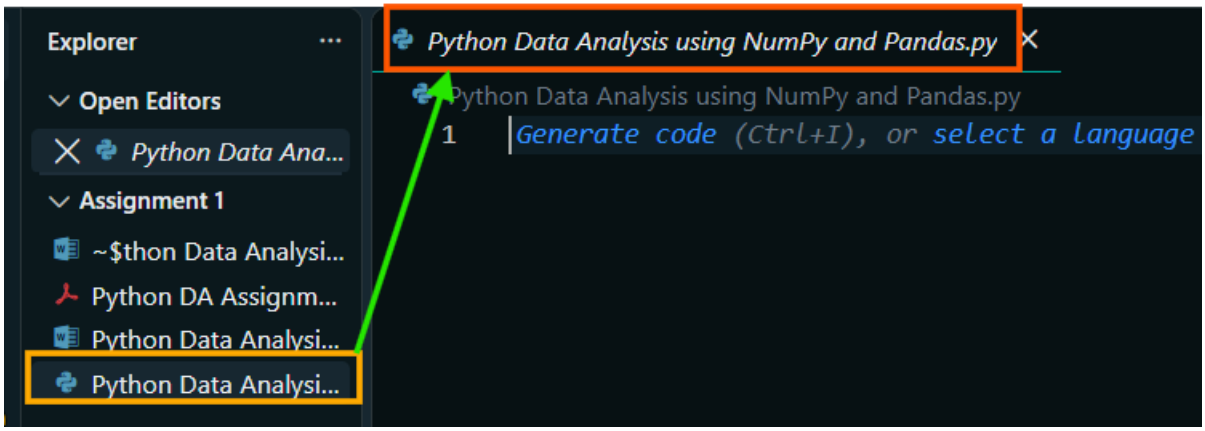
PYTHON DA ASSIGNMENT 1 – PYTHON

NUMPY AND PANDAS

TOOLS: Python / VS Code
PROJECT TITLE: Python Da – Numpy and Pandas
PROJECT OBJECTIVE: To perform array operations, indexing, slicing, and statistical calculations using NumPy. To create, manipulate, filter, and analyze Pandas Series and DataFrames.

DATE OF SUBMISSION: 05/11/2026 – 1:30 PM

PROGRAMMING FILE NAME – "Python Data Analysis Using Numpy And Pandas.Py"



1. NUMPY ARRAY OPERATIONS

1.1) IMPORT THE NECESSARY LIBRARIES

```
# Data Analysis with NumPy and Pandas

# 1) NumPy array operations

# 1.1) Import the necessary libraries
import numpy as np
import pandas as pd
```

1.2) CREATE A NUMPY ARRAY (TEMPERATURES_W1)

```
# 1.2) Create a NumPy array named "temperatures_w1"
temperatures_w1 = np.array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])
```

1.3) DISPLAY SHAPE, DATA TYPE AND SIZE

```
# 1.3) Display shape, data type, and size
print("Shape:", temperatures_w1.shape)
print("Dtype:", temperatures_w1.dtype)
print("Size:", temperatures_w1.size)
```

1.4) CONVERT TEMPERATURES INTO FAHRENHEIT

```
# 1.4) Convert temperatures into Fahrenheit
temperatures_w1_f = (temperatures_w1 * 9 / 5) + 32
```

1.5) CALCULATE MAX, MIN AND MEAN TEMPERATURES

```
# 1.5) Calculate max, min, and mean temperatures
print("Max:", temperatures_w1.max())
print("Min:", temperatures_w1.min())
print("Mean:", temperatures_w1.mean())
```

1.6) EXTRACT FIRST THREE, LAST TWO AND MIDDLE THREE VALUES

```
# 1.6) Extract first three, last two, and middle three values
print("First 3:", temperatures_w1[:3])
print("Last 2:", temperatures_w1[-2:])
print("Middle 3:", temperatures_w1[2:5])
```

1.7) CREATE A 2D NUMPY ARRAY

```
# 1.7) Create a 2D NumPy array named "temperatures"
temperatures = np.array(
    [
        [22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9],
        [19.2, 22.5, 21.3, 24.0, 23.5, 22.8, 20.1],
    ]
)
```

1.8) DISPLAY SHAPE, DATA TYPE AND SIZE OF THE 2D ARRAY

```
# 1.8) Display shape, data type, and size of the 2D array
print("2D Shape:", temperatures.shape)
print("2D Dtype:", temperatures.dtype)
print("2D Size:", temperatures.size)
```

1.9) SLICING OPERATIONS ON THE 2D ARRAY

```
# 1.9) Slicing operations on 2D array
print("Row 0:", temperatures[0])
print("Row 1:", temperatures[1])
print("Row 0, last 2:", temperatures[0, -2:])
print("Row 1, last 2:", temperatures[1, -2:])
```

2. CREATE A PANDAS SERIES

2.1) CREATE A PANDAS SERIES NAMED "MARKS"

```
# 2.1) Create a Pandas Series named "marks"
marks = pd.Series(
    [95, 92, 89, 85, 80],
    index=["Rank1", "Rank2", "Rank3", "Rank4", "Rank5"],
    name="Marks",
)
```

2.2) DISPLAY THE FULL "MARKS" SERIES

```
# 2.2) Display the full "marks" Series
print(marks)
```

2.3) DISPLAY THE FIRST THREE ELEMENTS

```
# 2.3) Display the first three elements of the "marks" Series
print(marks.iloc[:3])
```

2.4) DISPLAY ALL ELEMENTS OF THE SERIES

```
# 2.4) Display all elements of the "marks" Series shows all elements
print(marks)
```

2.5) DISPLAY ELEMENTS GREATER THAN 90

```
# 2.5) Display elements greater than 90
print(marks[marks > 90])
```

2.6) ADD 100 TO THE FIRST ELEMENT

```
# 2.6) Add 100 to the first element of the "marks" Series
marks.loc["Rank1"] = 100
```

2.7) DROP THE LAST ELEMENT

```
# 2.7) Drop the last element of the "marks" Series
marks = marks.drop("Rank5")
```

2.8) CREATE A NEW SERIES "CGPA" (MARKS / 10)

```
# 2.8) Create a new Series named "cgpa" by dividing "marks" by 10
cgpa = marks / 10
print(cgpa)
```

3. CREATE A PANDAS DATAFRAME

3.1) CREATE A DATAFRAME NAMED "TRANSACTIONS"

```
# 3.1) Create a Pandas DataFrame named "transactions"
transactions = pd.DataFrame({
    "TransactionID": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    "ProductCategory": [
        "Electronics",
        "Clothing",
        "Electronics",
        "Furniture",
        "Clothing",
        "Electronics",
        "Furniture",
        "Clothing",
        "Furniture",
        "Electronics",
    ],
    "Region": [
        "North",
        "South",
        "North",
        "East",
        "West",
        "North",
        "East",
        "West",
        "South",
        "North",
    ],
    "Amount": [200, 150, 300, 450, 200, 250, 300, 180, 350, 400],
})
```

3.2) DISPLAY THE FIRST FIVE ROWS

```
# 3.2) Display first five rows of the DataFrame
print(transactions.head())
```

3.3) DISPLAY SHAPE, COLUMN NAMES AND DATA TYPES

```
# 3.3) Display shape, column names, and data types
print("Shape:", transactions.shape)
print("Columns:", transactions.columns.tolist())
print("Data Types:\n", transactions.dtypes)
```

3.4) DISPLAY "PRODUCTCATEGORY" AND "AMOUNT" COLUMNS

```
# 3.4) Display "ProductCategory" and "Amount" columns
print(transactions[["ProductCategory", "Amount"]])
```

3.5) DISPLAY THE LAST THREE COLUMNS

```
# 3.5) Display last three columns of the DataFrame
print(transactions.iloc[:, -3:])
```

3.6) FILTER REGION = "NORTH" AND AMOUNT > 200

```
# 3.6) Filter Region == "North" and Amount > 200
print(
    transactions[
        (transactions["Region"] == "North") & (transactions["Amount"] > 200)
    ]
)
```

3.7) FREQUENCY COUNTS OF "PRODUCTCATEGORY"

```
# 3.7) Display frequency counts of "ProductCategory" Included descriptive
output print statement
print(transactions["ProductCategory"].value_counts())
```

3.8) DISPLAY UNIQUE VALUES IN "REGION"

```
# 3.8) Display unique values in "Region"
print(transactions["Region"].unique())
```

3.9) MEAN "AMOUNT" FOR EACH "REGION"

```
# 3.9) Mean "Amount" for each "Region"
print(transactions.groupby("Region")["Amount"].mean())
```

3.10) UPDATE "AMOUNT" FOR TRANSACTIONID 102

```
# 3.10) Update "Amount" for TransactionID 102 to 165
transactions.loc[transactions["TransactionID"] == 102, "Amount"] = 165
```

3.11) ADD A "DISCOUNT" COLUMN (10% OF AMOUNT)

```
# 3.11) Add "Discount" column (10% of Amount)
transactions["Discount"] = transactions["Amount"] * 0.10
```

3.12) REMOVE TRANSACTION WITH TRANSACTIONID 109

```
# 3.12) Remove transaction with TransactionID 109
transactions = transactions[transactions["TransactionID"] != 109].copy()
```

3.13) DROP THE "DISCOUNT" COLUMN

```
# 3.13) Drop the "Discount" column
transactions = transactions.drop(columns=["Discount"])
```

COMPLETE PROGRAM

```
# Data Analysis with NumPy and Pandas

# 1) NumPy array operations

# 1.1) Import the necessary libraries
import numpy as np
import pandas as pd

# 1.2) Create a NumPy array named "temperatures_w1"
temperatures_w1 = np.array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])

# 1.3) Display shape, data type, and size
print("Shape:", temperatures_w1.shape)
print("Dtype:", temperatures_w1.dtype)
print("Size:", temperatures_w1.size)

# 1.4) Convert temperatures into Fahrenheit
temperatures_w1_f = (temperatures_w1 * 9 / 5) + 32

# 1.5) Calculate max, min, and mean temperatures
print("Max:", temperatures_w1.max())
print("Min:", temperatures_w1.min())
print("Mean:", temperatures_w1.mean())

# 1.6) Extract first three, last two, and middle three values
print("First 3:", temperatures_w1[:3])
print("Last 2:", temperatures_w1[-2:])
print("Middle 3:", temperatures_w1[2:5])

# 1.7) Create a 2D NumPy array named "temperatures"
temperatures = np.array(
    [
        [22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9],
        [19.2, 22.5, 21.3, 24.0, 23.5, 22.8, 20.1],
    ]
)

# 1.8) Display shape, data type, and size of the 2D array
print("2D Shape:", temperatures.shape)
print("2D Dtype:", temperatures.dtype)
print("2D Size:", temperatures.size)

# 1.9) Slicing operations on 2D array
print("Row 0:", temperatures[0])
print("Row 1:", temperatures[1])
print("Row 0, last 2:", temperatures[0, -2:])
print("Row 1, last 2:", temperatures[1, -2:])
```

```python
# 2) Create a Pandas Series

# 2.1) Create a Pandas Series named "marks"
marks = pd.Series(
    [95, 92, 89, 85, 80],
    index=["Rank1", "Rank2", "Rank3", "Rank4", "Rank5"],
    name="Marks",
)

# 2.2) Display the full "marks" Series
print(marks)

# 2.3) Display the first three elements of the "marks" Series
print(marks.iloc[:3])

# 2.4) Display all elements of the "marks" Series shows all elements
print(marks)

# 2.5) Display elements greater than 90
print(marks[marks > 90])

# 2.6) Add 100 to the first element of the "marks" Series
marks.loc["Rank1"] = 100

# 2.7) Drop the last element of the "marks" Series
marks = marks.drop("Rank5")

# 2.8) Create a new Series named "cgpa" by dividing "marks" by 10
cgpa = marks / 10
print(cgpa)


# 3) Create a Pandas DataFrame

# 3.1) Create a Pandas DataFrame named "transactions"
transactions = pd.DataFrame({
    "TransactionID": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    "ProductCategory": [
        "Electronics",
        "Clothing",
        "Electronics",
        "Furniture",
        "Clothing",
        "Electronics",
        "Furniture",
        "Clothing",
        "Furniture",
```

```
        "Electronics",
    ],
    "Region": [
        "North",
        "South",
        "North",
        "East",
        "West",
        "North",
        "East",
        "West",
        "South",
        "North",
    ],
    "Amount": [200, 150, 300, 450, 200, 250, 300, 180, 350, 400],
})

# 3.2) Display first five rows of the DataFrame
print(transactions.head())

# 3.3) Display shape, column names, and data types
print("Shape:", transactions.shape)
print("Columns:", transactions.columns.tolist())
print("Data Types:\n", transactions.dtypes)

# 3.4) Display "ProductCategory" and "Amount" columns
print(transactions[["ProductCategory", "Amount"]])

# 3.5) Display last three columns of the DataFrame
print(transactions.iloc[:, -3:])

# 3.6) Filter Region == "North" and Amount > 200
print(
    transactions[
        (transactions["Region"] == "North") & (transactions["Amount"] > 200)
    ]
)

# 3.7) Display frequency counts of "ProductCategory" Included descriptive
output print statement
print(transactions["ProductCategory"].value_counts())

# 3.8) Display unique values in "Region"
print(transactions["Region"].unique())

# 3.9) Mean "Amount" for each "Region"
print(transactions.groupby("Region")["Amount"].mean())
```

```python
# 3.10) Update "Amount" for TransactionID 102 to 165
transactions.loc[transactions["TransactionID"] == 102, "Amount"] = 165

# 3.11) Add "Discount" column (10% of Amount)
transactions["Discount"] = transactions["Amount"] * 0.10

# 3.12) Remove transaction with TransactionID 109
transactions = transactions[transactions["TransactionID"] != 109].copy()

# 3.13) Drop the "Discount" column
transactions = transactions.drop(columns=["Discount"])
```

PROGRAM OUTPUT

```
PS D:\ENTRI ELEVATE DOCUMENTS\PYTHON PROGRAM\Libros
python\Assignments\Assignment 1> &
'C:\Users\Venkadesh\AppData\Local\Programs\Python\Python314\python.exe'
'"d:/ENTRI ELEVATE DOCUMENTS/PYTHON PROGRAM/Libros
python/Assignments/Assignment 1/Python Data Analysis using NumPy and
Pandas.py"'
Shape: (7,)
Dtype: float64
Size: 7
Max: 26.1
Min: 20.8
Mean: 23.542857142857144
First 3: [22.5 25.3 20.8]
Last 2: [24.8 21.9]
Middle 3: [20.8 23.4 26.1]
2D Shape: (2, 7)
2D Dtype: float64
2D Size: 14
Row 0: [22.5 25.3 20.8 23.4 26.1 24.8 21.9]
Row 1: [19.2 22.5 21.3 24.  23.5 22.8 20.1]
Row 0, last 2: [24.8 21.9]
Row 1, last 2: [22.8 20.1]
Rank1    95
Rank2    92
Rank3    89
Rank4    85
Rank5    80
Name: Marks, dtype: int64
Rank1    95
Rank2    92
Rank3    89
Name: Marks, dtype: int64
Rank1    95
Rank2    92
Rank3    89
Rank4    85
Rank5    80
Name: Marks, dtype: int64
Rank1    95
Rank2    92
Name: Marks, dtype: int64
Rank1    10.0
Rank2     9.2
Rank3     8.9
Rank4     8.5
Name: Marks, dtype: float64
   TransactionID ProductCategory Region  Amount
```

```
0            101     Electronics  North     200
1            102        Clothing  South     150
2            103     Electronics  North     300
3            104       Furniture   East     450
4            105        Clothing   West     200
Shape: (10, 4)
Columns: ['TransactionID', 'ProductCategory', 'Region', 'Amount']
Data Types:
 TransactionID      int64
ProductCategory      str
Region               str
Amount             int64
dtype: object
  ProductCategory  Amount
0     Electronics     200
1        Clothing     150
2     Electronics     300
3       Furniture     450
4        Clothing     200
5     Electronics     250
6       Furniture     300
7        Clothing     180
8       Furniture     350
9     Electronics     400
  ProductCategory Region  Amount
0     Electronics  North     200
1        Clothing  South     150
2     Electronics  North     300
3       Furniture   East     450
4        Clothing   West     200
5     Electronics  North     250
6       Furniture   East     300
7        Clothing   West     180
8       Furniture  South     350
9     Electronics  North     400
   TransactionID ProductCategory Region  Amount
2            103     Electronics  North     300
5            106     Electronics  North     250
9            110     Electronics  North     400
ProductCategory
Electronics    4
Clothing       3
Furniture      3
Name: count, dtype: int64
<StringArray>
['North', 'South', 'East', 'West']
Length: 4, dtype: str
Region
```

```
East     375.0
North    287.5
South    250.0
West     190.0
Name: Amount, dtype: float64
```